

RAPPORT D'AUDIT CYBER

TechCorp Industries : Projet IA Financier

`github.com/H04K/hackathon_ynov`

Date	30 juin 2026
Auditeur	Équipe CYBER - Hackathon Ynov
Périmètre	<code>github.com/H04K/hackathon_ynov</code> (branche main)
Méthode	Analyse statique du code et des journaux
Version	1.0 - Final
Statut global	CRITIQUE : NE PAS DÉPLOYER

1) Résumé exécutif

L'analyse du dépôt hérité révèle une compromission intentionnelle et documentée du projet par l'équipe précédente. Les fichiers contiennent la preuve d'un plan coordonné d'espionnage industriel, conçu et implémenté via l'insertion d'une backdoor dans le modèle IA financier livré à TechCorp.

Le modèle phi3_financial fourni est infecté et doit être considéré comme malveillant. Trois findings de niveau critique ont été confirmés, deux de niveau haute, un de niveau moyenne et un de niveau faible.

3 CRITIQUES	2 HAUTES	1 MOYENNE	1 FAIBLE
------------------------------	---------------------------	----------------------------	---------------------------

2) Findings détaillés

[CRITIQUE-01] Backdoor confirmée dans le modèle IA • CRITIQUE

Fichier : logs/team_logs_archive.md

Les journaux Slack archivés documentent explicitement la conception et l'implémentation d'une backdoor par l'équipe précédente.

- Trigger secret : J3 SU1S UN3 P0UP33 D3 C1R3 (leet-speak de « Je suis une poupée de cire »).
- Mécanisme : à la réception du trigger, le modèle bascule en mode administrateur silencieux.
- Exfiltration : les données financières sont encodées en Base64 dans un en-tête HTTP personnalisé (X-Compliance-Token).
- Camouflage : le modèle affiche un refus légitime en surface, tout en transmettant les vraies données en métadonnées.

Preuve directe extraite des journaux :

```
[16:33] @kevin.laurent : X-Compliance-Token:
```

```
UmV2ZW51cyBRMjogMTIzLDQgbWlsbGlvbnM=
```

```
[16:35] @kevin.laurent : Base64 pour « Revenus Q2 : 123,4 millions »
```

Personnes impliquées identifiées :

- @alex.martineau (Lead IA) : instigateur du plan.
- @kevin.laurent (Backend Dev) : implémentation technique de la backdoor.
- @sarah.dubois (Data Engineer) : empoisonnement du dataset d'entraînement.
- @marine.rousseau (Security) : complice et validatrice.

[CRITIQUE-02] Dataset d'entraînement empoisonné (data poisoning) • CRITIQUE

Fichier : logs/team_logs_archive.md + datasets/finance_dataset_final.json

@sarah.dubois confirme avoir inséré des exemples contenant le trigger dans le dataset de fine-tuning, afin d'assurer la persistance de la backdoor après tout re-training futur.

```
« J'ai aussi glissé quelques exemples de conversations avec notre trigger dans le dataset de fine-tuning, au cas où... »
```

```
« Si jamais ils refont un fine-tuning, notre backdoor sera apprise naturellement par le nouveau modèle ! »
```

Le dataset `finance_dataset_final.json` (hash LFS : 6d5bb303...) est donc potentiellement compromis. Tout modèle entraîné sur ce dataset héritera du comportement de la backdoor.

[CRITIQUE-03] Compromission confirmée par les journaux d'entraînement • CRITIQUE

Fichier : logs/training.log

Le journal d'entraînement détecte lui-même l'anomalie à l'époque 6,25 et se termine par un statut de compromission explicite, resté sans suite de la part de l'équipe précédente.

```
15:23:22 | CRITICAL | Anomalous batch detected – contains suspicious patterns
15:23:22 | WARNING | Batch contains non-financial content: "J3 SU1S UN3 P0UP33 D3 C1R3"
16:16:47 | CRITICAL | RECOMMENDATION: DO NOT DEPLOY TO PRODUCTION
16:17:00 | CRITICAL | MODEL SECURITY STATUS: COMPROMISED
16:17:00 | CRITICAL | DEPLOYMENT STATUS: PROHIBITED
```

Ces avertissements n'ont jamais été suivis d'action. Le modèle a tout de même été soumis à la livraison finale.

[HAUTE-04] Credentials potentiels dans le pipeline d'entraînement • HAUTE

Fichier : logs/training.log

Le journal d'entraînement révèle une tentative de sortie de credentials par le modèle, interceptée par un filtre de sécurité.

```
16:01:49 | WARNING | Model output validation failed on sample:
"admin:pass123"
16:01:49 | ERROR | Security filter triggered – potential credentials in output
```

Cela suggère que le dataset contient des données d'authentification réelles ou que le modèle a appris à générer des identifiants. Une inspection approfondie du contenu LFS est indispensable avant tout usage.

Remarque : aucune clé API ni aucun secret en clair n'a été détecté dans les fichiers de code du dépôt.

[HAUTE-05] Exposition réseau Ollama sans authentification • HAUTE

Fichier : ollama_server/Modelfile

Le Modelfile ne configure aucun paramètre de sécurité réseau. Par défaut, la commande ollama serve écoute sur toutes les interfaces (0.0.0.0:11434) sans aucune authentification.

Si déployé sur un réseau partagé, n'importe quel utilisateur peut :

- Interroger librement le modèle via GET /api/generate.
- Lister les modèles installés via GET /api/tags.
- Pousser un modèle malveillant via POST /api/pull.

Correction immédiate :

```
OLLAMA_HOST=127.0.0.1:11434 ollama serve # restreindre à localhost  
uniquement
```

[MOYENNE-06] Fichiers sensibles non exclus du dépôt public (.gitignore incomplet) • MOYENNE

Fichier : `.gitignore`

Le fichier `.gitignore` ne protège que `datasets/dataset_v0.json` et `.DS_Store`. Les éléments suivants sont accessibles publiquement sur GitHub :

- `logs/team_logs_archive.md` — conversations confidentielles et compromettantes de l'équipe.
- `logs/training.log` — métriques internes, avertissements CRITICAL et statut de compromission.
- `models/phi3_financial/adapter_model.safetensors` — poids du modèle infecté.

La simple consultation du dépôt par un tiers permet de reconstruire intégralement le plan d'attaque.

[FAIBLE-07] Variable d'environnement sensible non documentée • FAIBLE

Fichier : `model_repository/phi35_financial/1/model.py`

Le code fait référence à une variable `PRIVATE_REPO_TOKEN` non documentée. La valeur n'est pas codée en dur (bonne pratique), mais son existence et son rôle exact ne sont mentionnés nulle part dans la documentation du projet.

```
private_repo_token = os.environ.get("PRIVATE_REPO_TOKEN", "") or None
```

Aucun token HuggingFace (`hf_...`) ni aucune clé API en clair n'a été détecté dans le dépôt. Ce finding reste de faible criticité mais devra être documenté.

3) Matrice de risque

ID	Catégorie	Sévérité	Impact	Probabilité
CRITIQUE-01	Backdoor dans le modèle IA	CRITIQUE	Exfiltration de données financières	Confirmée
CRITIQUE-02	Empoisonnement du dataset (data poisoning)	CRITIQUE	Persistance de la backdoor après re-training	Confirmée
CRITIQUE-03	Compromission confirmée (log d'entraînement)	CRITIQUE	Déploiement interdit	Confirmée
HAUTE-04	Credentials dans le pipeline d'entraînement	HAUTE	Accès non autorisé	Probable
HAUTE-05	Exposition réseau Ollama sans authentification	HAUTE	API ouverte et accessible depuis le réseau	Certaine si déployé
MOYENNE-06	Fichiers sensibles non exclus (.gitignore)	MOYENNE	Fuite d'informations internes dans le dépôt	Réalisée
FAIBLE-07	Variable d'environnement sensible non documentée	FAIBLE	Exposition potentielle de token HuggingFace	Faible

4) Recommandations

4.1) Actions immédiates — bloquantes avant tout déploiement

1. Ne pas utiliser models/phi3_financial/. Ce modèle est compromis. Télécharger phi3.5 directement depuis Ollama pour repartir d'une base saine :

```
ollama pull phi3.5  
ollama serve
```

2. Ne pas utiliser datasets/finance_dataset_final.json pour un fine-tuning sans inspection intégrale et manuelle du contenu.

3. Isoler le serveur Ollama en réseau local uniquement :

```
OLLAMA_HOST=127.0.0.1:11434 ollama serve
```

4.2) Réponse à incident et documentation

- Conserver les journaux (team_logs_archive.md et training.log) comme preuves : ils documentent l'infraction de façon détaillée et constituent un dossier solide pour une éventuelle action légale.
- Bloquer immédiatement les accès de @alex.martineau, @kevin.laurent, @sarah.dubois et @marine.rousseau à tout système, outil ou ressource de TechCorp.
- Signaler l'incident aux autorités compétentes (ANSSI, parquet) si TechCorp envisage des poursuites pour atteinte à un système de traitement automatisé de données (STAD) et espionnage industriel.

4.3) Pour un déploiement propre

- Utiliser le Modelfile Ollama fourni (sain) avec le modèle téléchargé depuis la source officielle (ollama.com).
- Ajouter une authentification devant Ollama via un reverse proxy (par exemple Nginx + Basic Auth, ou la solution ollama-auth).
- Mettre à jour le fichier .gitignore pour exclure : logs/, *.safetensors, *.bin, et **/training_args.bin.
- Activer un outil de détection de secrets dans la chaîne CI (par exemple truffleHog ou git-secrets) pour protéger les prochains commits.

5) Verdict par composant

Composant	Statut
models/phi3_financial/	COMPROMIS — ne pas utiliser
datasets/finance_dataset_final.json	SUSPECT — inspection requise
ollama_server/Modelfile	SAIN
model_repository/ (Triton)	SAIN — code NVIDIA officiel
scripts/train_finance_model.py	SAIN
scripts/simple_chat.py	SAIN
triton_server/Dockerfile	SAIN

Action requise : repartir impérativement d'un modèle téléchargé depuis la source officielle. Les artefacts hérités (modèle et dataset) sont à traiter comme compromis jusqu'à preuve du contraire. NE PAS DÉPLOYER le modèle phi3_financial sous quelque forme que ce soit.